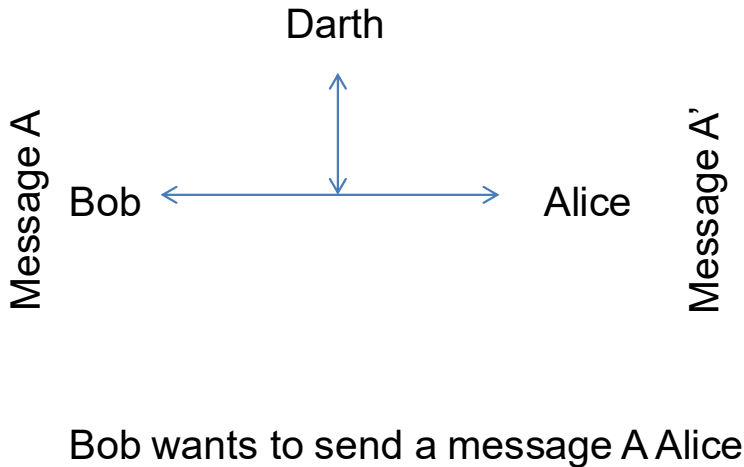


Public-Key Cryptography and RSA

Private-Key Cryptography

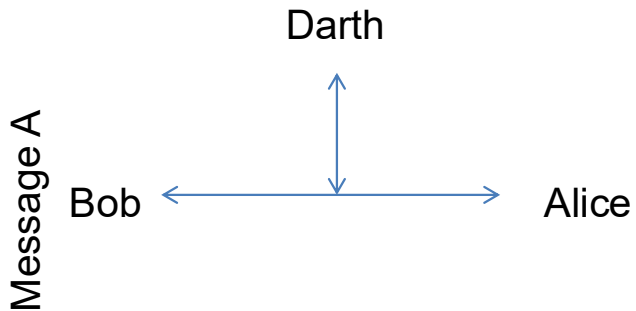
- traditional **private/secret/single key** cryptography uses **one** key
- shared by both sender and receiver
- if this key is disclosed communications are compromised
- also is **symmetric**, parties are equal
- hence does not protect sender from receiver forging a message & claiming is sent by sender

Challenges To Be Considered

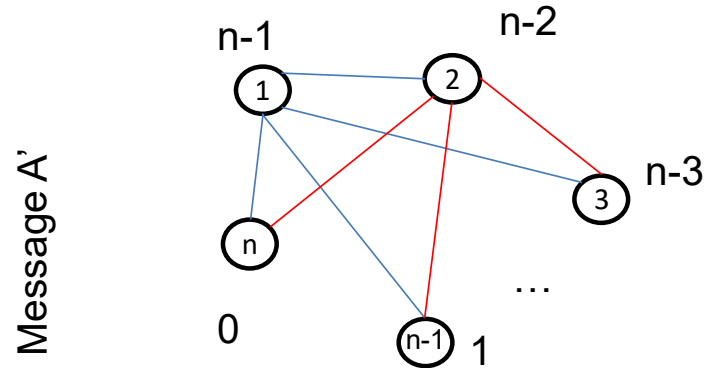


- How to keep message A as a secret from Darth?
- How does Alice know the message is not modified during transmission?
- How does Alice know the message comes from A?
- How can Bob ensure Alice does not deny receiving the message?
- How to avoid replay attack?
- How to keep the message fresh?
- More ...

Symmetric Cipher: Confidentiality



- Two users, key(s) required
 - 1 key

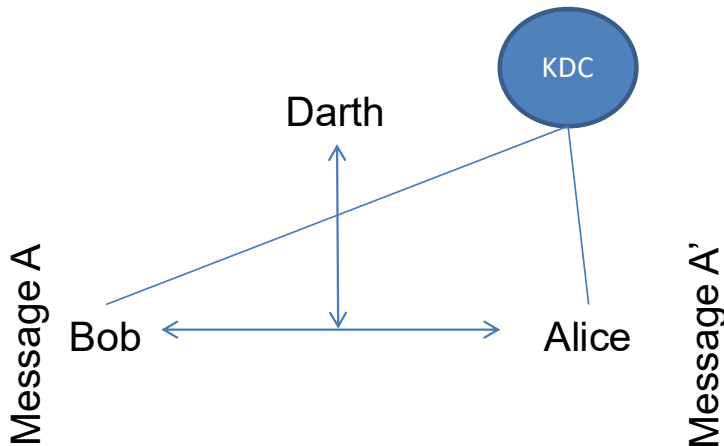


- n users, key(s) required
 - $0+1+2+\dots+(n-1)=n(n-1)/2$

Challenges: how to securely distribute $n(n-1)/2$ keys?

Symmetric Cipher: Authentication

The key is to create a signature for the message to be sent.



- Depends on a trusted party, e.g., Key Distribution Center (KDC)
- does not protect sender from receiver forging a message & claiming is sent by sender

Public-Key Cryptography

- probably most significant advance in the 3000 year history of cryptography
- uses **two** keys – a public & a private key
- **asymmetric** since parties are **not** equal
- uses clever application of number theoretic concepts to function
- complements **rather than** replaces private key crypto

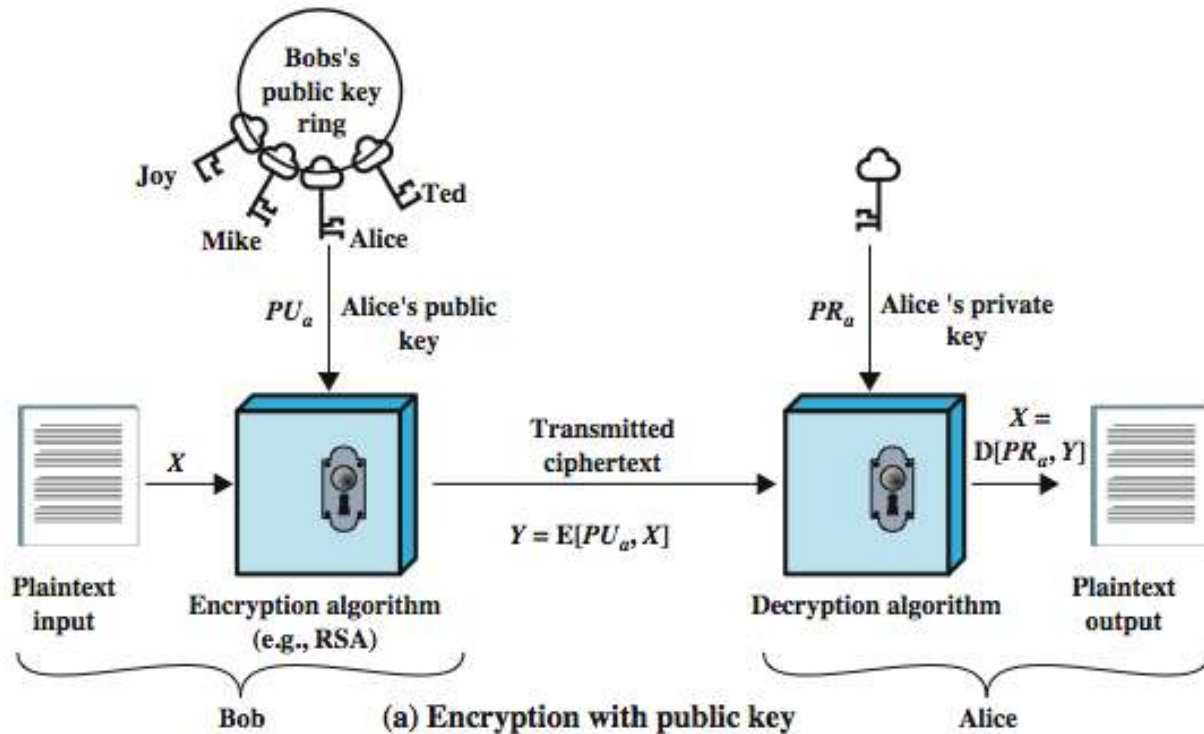
Why Public-Key Cryptography?

- developed to address two key issues:
 - **key distribution** – how to have secure communications in general without having to trust a KDC with your key
 - **digital signatures** – how to verify a message comes intact from the claimed sender
- public invention due to Whitfield Diffie & Martin Hellman at Stanford Uni in 1976
 - It was revealed in 1997 that James H. Ellis, Clifford Cocks, and Malcolm J. Williamson of GCHQ, the British signals intelligence agency, had previously shown in 1969 how public-key cryptography could be achieved.

Public-Key Cryptography

- **public-key/two-key/asymmetric** cryptography involves the use of **two** keys:
 - a **public-key**, which may be known by anybody, and can be used to **encrypt messages**, and **verify signatures**
 - a related **private-key**, known only to the recipient, used to **decrypt messages**, and **sign (create) signatures**
- **infeasible to determine private key from public**
- is **asymmetric** because
 - those who encrypt messages or verify signatures **cannot** decrypt messages or create signatures

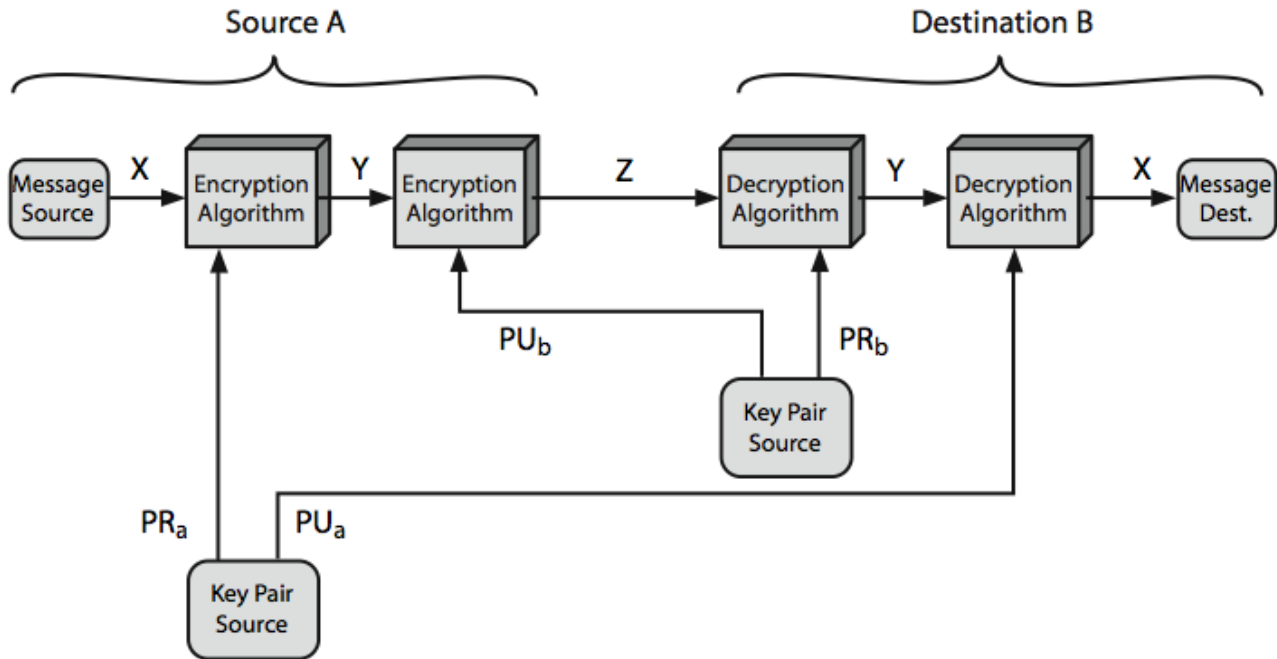
Public-Key Cryptography



Symmetric vs Public-Key

Conventional Encryption	Public-Key Encryption
<i>Needed to Work:</i> 1. The same algorithm with the same key is used for encryption and decryption. 2. The sender and receiver must share the algorithm and the key. <i>Needed for Security:</i> 1. The key must be kept secret. 2. It must be impossible or at least impractical to decipher a message if no other information is available. 3. Knowledge of the algorithm plus samples of ciphertext must be insufficient to determine the key.	<i>Needed to Work:</i> 1. One algorithm is used for encryption and decryption with a pair of keys, one for encryption and one for decryption. 2. The sender and receiver must each have one of the matched pair of keys (not the same one). <i>Needed for Security:</i> 1. One of the two keys must be kept secret. 2. It must be impossible or at least impractical to decipher a message if no other information is available. 3. Knowledge of the algorithm plus one of the keys plus samples of ciphertext must be insufficient to determine the other key.

Public-Key Cryptosystems



Public-Key Applications

- can classify uses into 3 categories:
 - **encryption/decryption** (provide secrecy)
 - **digital signatures** (provide authentication)
 - **key exchange** (of session keys)
- some algorithms are suitable for all uses, others are specific to one

Algorithm	Encryption/Decryption	Digital Signature	Key Exchange
RSA	Yes	Yes	Yes
Elliptic Curve	Yes	Yes	Yes
Diffie-Hellman	No	No	Yes
DSS	No	Yes	No

Public-Key Requirements

- Public-Key algorithms rely on two keys where:
 - it is computationally infeasible to find decryption key knowing only algorithm & encryption key
 - it is computationally easy to en/decrypt messages when the relevant (en/decrypt) key is known
 - either of the two related keys can be used for encryption, with the other used for decryption (for some algorithms)
- these are formidable requirements which only a few algorithms have satisfied

Public-Key Requirements

- need a trapdoor one-way function
- one-way function has
 - $Y = f(X)$ easy
 - $X = f^{-1}(Y)$ infeasible
- a trap-door one-way function has
 - $Y = f_k(X)$ easy, if k and X are known
 - $X = f_k^{-1}(Y)$ easy, if k and Y are known
 - $X = f_k^{-1}(Y)$ infeasible, if Y known but k not known
- a practical public-key scheme depends on a suitable trap-door one-way function

Security of Public Key Schemes

- like private key schemes brute force **exhaustive search** attack is always theoretically possible
- but keys used are too large (>512bits)
- security relies on a **large enough** difference in difficulty between **easy** (en/decrypt) and **hard** (cryptanalyse) problems
- more generally the **hard** problem is known, but is made hard enough to be impractical to break
- requires the use of **very large numbers**
- hence is **slow** compared to private key schemes

RSA

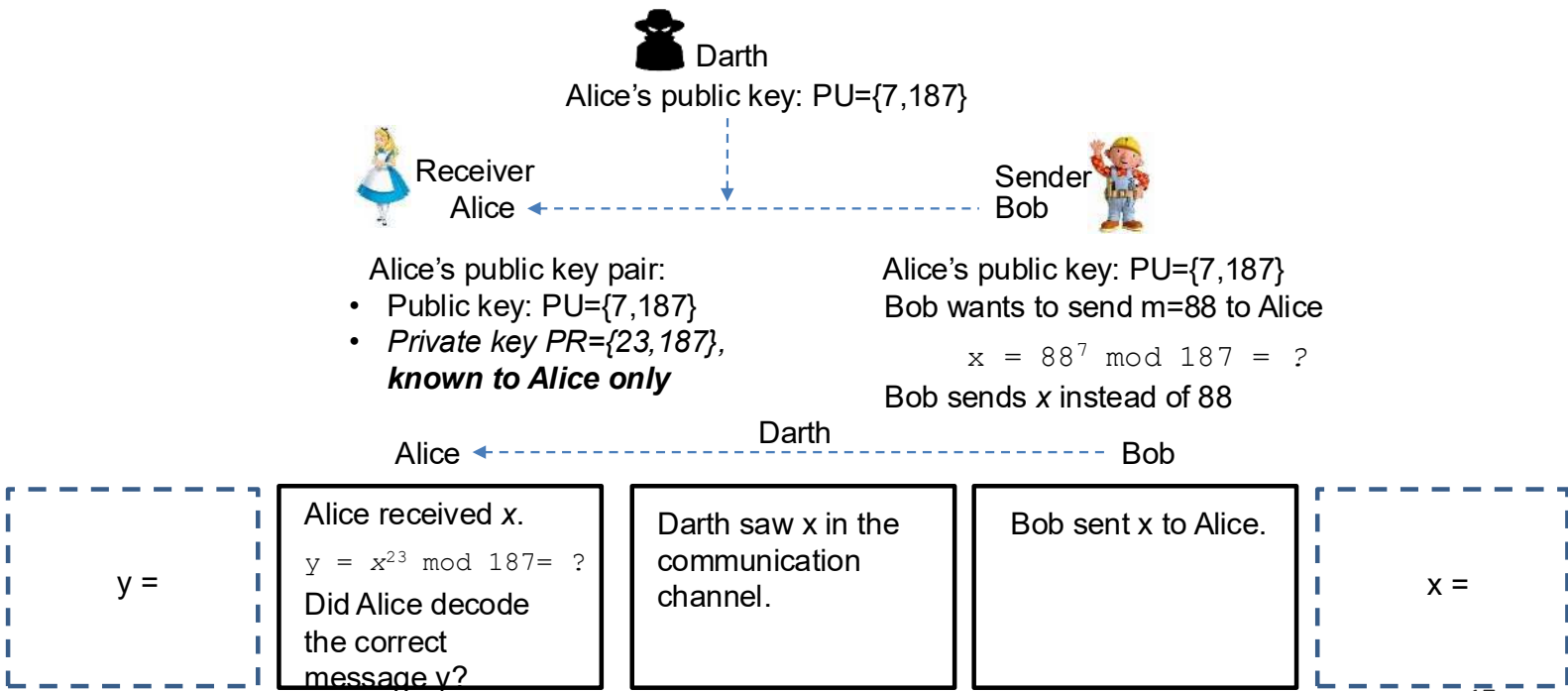
- by Rivest, Shamir & Adleman of MIT in 1977
- best known & widely used public-key scheme
- based on exponentiation in a finite (Galois) field over integers modulo a prime
 - nb. exponentiation takes $O((\log n)^3)$ operations (easy)
- uses large integers (eg. 1024 bits)
- security due to cost of factoring large numbers
 - nb. factorization takes $O(e^{\log n \log \log n})$ operations (hard)

RSA Example

Designed by Ron Rivest, Adi Shamir, and Leonard Adleman in 1977.

RSA Encryption/Decryption

- to encrypt a message M the sender:
 - obtains **public key** of recipient $PU=\{e, n\}$
 - computes: $C = M^e \bmod n$, where $0 \leq M < n$
- to decrypt the ciphertext C the owner:
 - uses their **private key** $PR=\{d, n\}$
 - computes: $M = C^d \bmod n$



RSA En/decryption

- to encrypt a message M the sender:
 - obtains **public key** of recipient $PU = \{e, n\}$
 - computes: $C = M^e \bmod n$, where $0 \leq M < n$
- to decrypt the ciphertext C the owner:
 - uses their private key $PR = \{d, n\}$
 - computes: $M = C^d \bmod n$
- note that the message M must be smaller than the modulus n (block if needed)

RSA Key Setup

- each user generates a public/private key pair by:
- selecting two large primes at random: p, q
- computing their system modulus $n = p \cdot q$
 - note $\phi(n) = (p-1)(q-1)$
- selecting at random the encryption key e
 - where $1 < e < \phi(n)$, $\gcd(e, \phi(n)) = 1$
- solve following equation to find decryption key d
 - $e \cdot d = 1 \pmod{\phi(n)}$ and $0 \leq d \leq n$
- publish their public encryption key: $PU = \{e, n\}$
- keep secret private decryption key: $PR = \{d, n\}$

Why RSA Works

- because of Euler's Theorem:
 - $a^{\phi(n)} \bmod n = 1$ where $\gcd(a, n) = 1$
- in RSA have:
 - $n = p \cdot q$
 - $\phi(n) = (p-1)(q-1)$
 - carefully chose e & d to be inverses mod $\phi(n)$
 - hence $e \cdot d = 1 + k \cdot \phi(n)$ for some k
- hence :
$$\begin{aligned} C^d &= M^{e \cdot d} = M^{1+k \cdot \phi(n)} = M^1 \cdot (M^{\phi(n)})^k \\ &= M^1 \cdot (1)^k = M^1 = M \bmod n \end{aligned}$$

RSA Example - Key Setup

1. Select primes: $p=17$ & $q=11$
2. Calculate $n = pq = 17 \times 11 = 187$
3. Calculate $\phi(n) = (p-1)(q-1) = 16 \times 10 = 160$
4. Select e : $\gcd(e, 160) = 1$; choose $e=7$
5. Determine d : $de=1 \pmod{160}$ and $d < 160$
Value is $d=23$ since $23 \times 7 = 161 = 1 \times 160 + 1$
6. Publish public key $PU = \{7, 187\}$
7. Keep secret private key $PR = \{23, 187\}$

RSA Example - En/Decryption

- sample RSA encryption/decryption is:
 - given message $M = 88$ (nb. $88 < 187$)
 - Publish public key $PU = \{ 7, 187 \}$
 - Private key $PR = \{ 23, 187 \}$
 - encryption:
- $$C = 88^7 \bmod 187 = 11$$
- decryption:

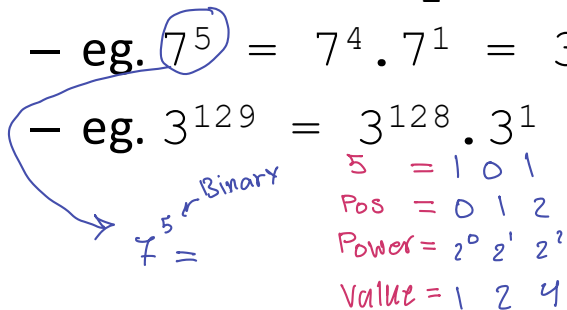
$$M = 11^{23} \bmod 187 = 88$$

Exponentiation

- can use the Square and Multiply Algorithm
- a fast, efficient algorithm for exponentiation
- concept is based on repeatedly squaring base
- and multiplying in the ones that are needed to compute the result
- look at binary representation of exponent
- only takes $O(\log_2 n)$ multiples for number n

– eg. $7^5 = 7^4 \cdot 7^1 = 3 \cdot 7 = 10 \pmod{11}$

– eg. $3^{129} = 3^{128} \cdot 3^1 = 5 \cdot 3 = 4 \pmod{11}$



5	=	1	0	1
Pos	=	0	1	2
Power	=	2^0	2^1	2^2
Value	=	1	2	4

Exponentiation

Algorithm for Computing $a^b \bmod n$

```
c = 0; f = 1
```

```
for i = k downto 0
```

The integer b is expressed as a binary number $b_k b_{k-1} \dots b_0$

```
do c = 2 x c
```

```
    f = (f x f) mod n
```

```
    if  $b_i == 1$ 
```

```
        then c = c + 1
```

```
            f = (f x a) mod n
```

```
return f
```


Exponentiation Example

$$\begin{aligned} C &= 2 \times C \\ F &= F \times F \bmod n \end{aligned} \quad \left. \begin{array}{l} \text{إذا } C \\ = 1 \\ \text{أكمل باقي} \\ \text{المعادلة} \end{array} \right\}$$

$$\begin{aligned} C &= C + 1 \\ F &= (F \times a) \bmod n \end{aligned}$$

$a^b \bmod n$

$$\begin{aligned} C &= 2 \times C \\ F &= F \times F \bmod n \\ C &= C + 1 \end{aligned}$$

$a = 7$

$b = 560 = 1000110000$

$n = 561$

من اعلى n
ب 1001

الضمان

أبعد
من

من اعلى
ب 1001

$c = 0$

$f = 1$

Position

من اعلى
ب 1001

i	9	8	7	6	5	4	3	2	1	0
b_i	1	0	0	0	1	1	0	0	0	0
c	1	2	4	8	17	35	70	140	280	560
f	7	49	157	526	160	241	298	166	67	1

$$f = 1 \bmod 561$$

$$f = 1 \times 7 \bmod 561$$

$$f = 7 \times 7 \bmod 561$$

$$f = 49 \times 49 \bmod 561$$

Efficient Encryption

- encryption uses exponentiation to power e
- hence if e small, this will be faster
 - often choose $e=65537$ ($2^{16}+1$)
 - also see choices of $e=3$ or $e=17$
- but if e too small (eg $e=3$) can attack
 - using Chinese remainder theorem & 3 messages with different moduli
- if e fixed must ensure $\gcd(e, \phi(n)) = 1$
 - ie reject any p or q not relatively prime to e

Efficient Decryption

- decryption uses exponentiation to power d
 - this is likely large, insecure if not
- can use the Chinese Remainder Theorem (CRT) to compute mod p & q separately. then combine to get desired answer
 - approx 4 times faster than doing directly
- only owner of private key who knows values of p & q can use this technique

RSA Key Generation

- users of RSA must:
 - determine two primes at random - p , q
 - select either e or d and compute the other
- primes p , q must not be easily derived from modulus $n=p \cdot q$
 - means must be sufficiently large
 - typically guess and use probabilistic test
- exponents e , d are inverses, so use Inverse algorithm to compute the other

RSA Security

- possible approaches to attacking RSA are:
 - brute force key search - infeasible given size of numbers
 - mathematical attacks - based on difficulty of computing $\phi(n)$, by factoring modulus n
 - timing attacks - on running of decryption
 - chosen ciphertext attacks - given properties of RSA

Factoring Problem

المسألة كلها
تقريباً على
رقمين
p and q

- mathematical approach takes 3 forms:
 - factor $n=p \cdot q$, hence compute $\phi(n)$ and then d
 - determine $\phi(n)$ directly and compute d
 - find d directly
- currently believe all equivalent to factoring
 - have seen slow improvements over the years
 - as of May-05 best is 200 decimal digits (663) bit with LS
 - biggest improvement comes from improved algorithm
 - cf QS to GNFS to LS
 - currently assume 1024-2048 bit RSA is secure
 - ensure p, q of similar size and matching other constraints

*this slide
just for
knowing*

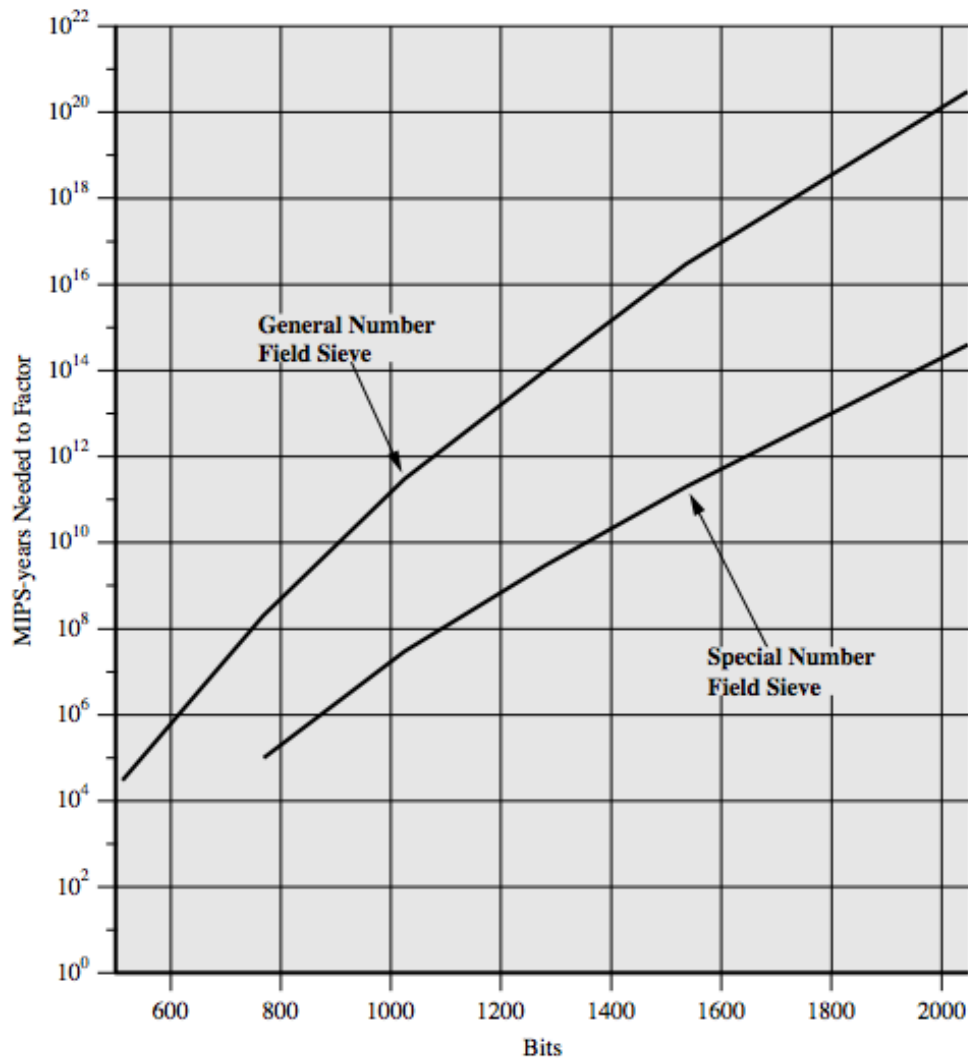
Progress in Factoring

Number of Decimal Digits	Approximate Number of Bits	Date Achieved	<u>MIPS-years</u>	Algorithm
<u>100</u>	<u>332</u>	April 1991	<u>7</u>	quadratic sieve
110	365	April 1992	75	quadratic sieve
120	398	June 1993	830	quadratic sieve
129	428	April 1994	5000	quadratic sieve
130	431	April 1996	1000	generalized number field sieve
140	465	February 1999	2000	generalized number field sieve
155	512	August 1999	8000	generalized number field sieve
160	530	April 2003	—	Lattice sieve
174	576	December 2003	—	Lattice sieve
200	663	May 2005	—	Lattice sieve

MIPS-year is equal to the number of instructions executed during one year of computing at one million instructions per second.

ignore

Progress in Factoring



Timing Attacks

- developed by Paul Kocher in mid-1990's
- exploit timing variations in operations
 - eg. multiplying by small vs large number
 - or IF's varying which instructions executed
- infer operand size based on time taken
- RSA exploits time taken in exponentiation
- countermeasures
 - ✂ use constant exponentiation time
 - ✂ add random delays
 - ✂ blind values used in calculations

Chosen Ciphertext Attacks

- RSA is vulnerable to a Chosen Ciphertext Attack (CCA)
- attackers chooses ciphertexts & gets decrypted plaintext back
- choose ciphertext to exploit properties of RSA to provide info to help cryptanalysis
- can counter with random pad of plaintext
- or use Optimal Asymmetric Encryption Padding (OASP)

Summary

- have considered:
 - principles of public-key cryptography
 - RSA algorithm, implementation, security